Logistic Regression in R

Kachun Huang

2025-02-20

Logistic Regression in R

The logistic distribution constrains the estimated probabilities to lie between 0 and 1.

The estimated probability is:

$$p = \frac{1}{1 + e^{-\alpha - \beta X}}$$

- If you let $\alpha + \beta X = 0$, then $p = 0.5$
- as $\alpha + \beta X = 0$ gets really big, p approaches to 1
- as $\alpha + \beta X = 0$ gets really small, p approaches to 0

Logistic Transformation

the **logistic model** (or **logit model**) is a statistical model that models the log-odds of an event as a linear combination of one or more independent variables.

$$log(\frac{p}{1-p}) = \alpha + \beta X$$

where,

- $log(\frac{p}{1-p})$ is the log odds range from $(-\infty, \infty)$

Perform fitting in R

The `glm()` function fits generalized linear models, a class of models that includes logistic regression.

The syntax of the `glm()` function is similar to that of `lm()`,except that we must pass in the argument `family=binomial` in order to tell R to run a logistic regression rather than some other type of generalized linear model.

syntax:

```
fit <- glm(
  response ~ pred1 + pred2 + ...,
  family = binomial,
  data = data_name
)
```

Coefficients

There are 2 ways to access the coefficients of the logistic model:

- `coef()`- access just the coefficients of the fitted model
- `summary()` - access the full summary information of the fitted model

syntax:

```
#with coef()
coef(fit)
```

```
#with summary()
summary(fit)$coef

#If just need the estimated parameter
summary(fit)$coef[,4]
```

Prediction

The `predict()` function can be used to make prediction using the fitted model, given values of the predictors.

The `type="response"` option tells R to output probabilities of the form $P(Y = 1|X)$ to other information such as the logit.

syntax:

```
# find Y hat of each x
pred_fit <- predict(fit, type="repsonse)
```

```
new_data <- dataframe(column1 = ..., column2= ..., column3=...)
# find Y hat of a given X
pred_fit <- predict(fit, new_data, type="repsonse")
```

Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model by comparing predicted vs. actual values. It provides a breakdown of correct and incorrect predictions across different classes.

When positive class = "0" Detect negative

| Actual/Predicted | Positive (Actual 0) | Negative (Actual 1) |

| Positive (Predicted 0) | **True Positive(TP)** | **False Positive(FP)** |

| Negative (Predicted 1) | **False Negative(FN)** | **True Negative (TN)** |

When positive class = "1" Detect positive

| Actual/Predicted | Negative (Actual 0) | Positive (Acutal 1) |

| Negative (Predicted 0) | **True Negative (TN)** | **False Negative(FN)** |

| Positive (Predicted 1) | **False Positive(FP)** | **True Positive(TP)** |

Build from scratch:

syntax:

```
# For positive class = "0" and negative class = "1"
#create predicted class
predicted.classes <- ifelse(pred_fit > 0.5, "neg", "pos")

#create confusion matrix
table(predicted.classes,data$response)
```

```
# For positive class = "1" and negative class = "0"
#create predicted class
predicted.classes <- ifelse(pred_fit > 0.5, "pos", "neg")

#create confusion matrix
table(predicted.classes,data$response)
```

Built-in function:

syntax:

```
# For positive class = "0" and negative class = "1"
confusionMatrix(factor(predicted.classes), factor(data$response), positive ="0")
```

```
# For positive class = "1" and negative class = "0"
confusionMatrix(factor(predicted.classes), factor(data$response), positive ="1")
```

Model Accuracy

To evaluate the overall model accuracy, we need to calculate

$P(\text{Predicted Correctly}) = \frac{TP+TN}{TP+TN+FP+FN}$

- **(# of sample predicted Y=1 correctly + # of sample predicted Y=1 correctly)** is corresponding to the **sum of the diagonal of confusion matrix**

To do this efficiently in R:

```
mean(predicted.classes == data$response)
```

Sensitivity

Sensitivity (also called the **true positive rate**, or the **recall** in some fields) measures the proportion of actual positives which are correctly identified as such (e.g., the percentage of sick people who are correctly identified as having the condition), and is complementary to the false negative rate.

$Sensitivity = P(\text{true positive}|\text{correct}) = \frac{TP}{TP+FN}$

To do this efficiently in R, we can access from the confusionMatrix()

Specificity

Specificity (also called the **true negative rate**) measures the proportion of negatives which are correctly identified as such (e.g., the percentage of healthy people who are correctly identified as not having the condition), and is complementary to the false positive rate.

$Specificity = P(\text{true negative}|\text{correct}) = \frac{TN}{TN+FP}$

Precision

precision is **a metric that measures how well a model predicts positive outcomes**. It's calculated by dividing the number of true positives by the total number of positive predictions.

$Precision = P(\text{true positiv}|\text{ actaully positive}) = \frac{TP}{TP+FP}$

F1

the **F1 score** is a measure of predictive performance. It is calculated from the precision and recall of the test, where the precision is the number of true positive results divided by the number of all samples predicted to be positive, including those not identified correctly, and the recall is the number of true positive results divided by the number of all samples that should have been identified as positive.

$$F_1 = 2 * \frac{Precision * Recall}{Precision + Recall}$$

Compare models

One of the most important differences between logistic regression and linear regression is in how we compare models.

- For linear regression, we looked at how the adjusted R2 changed. If there was a significant increase when we added another variable (or interaction) then we thought the model had improved.
- For logistic regression, we use likelihood-ratio test

```
#import lmtest library
library(lmtest)

#use likelihood test
lrtest(submodel, fullmodel)
```

Example

```
library(mlbench)
```

```
## Warning: package 'mlbench' was built under R version 4.3.3
```

```
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   3.5.1     v tibble    3.2.1
## v lubridate 1.9.3     v tidyr     1.3.0
## v purrr     1.0.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

Data Import

```
#import data from mlbench
data("PimaIndiansDiabetes2", package= "mlbench")
```

Data Description:

The 9 variables of the pimaindiansdiabetes2 tbl data frame are the following ones:

- **pregnant**- Number of times pregnant (numeric);
- **glucose**- Plasma glucose concentration a 2 hours in an oral glucose tolerance test (numeric);
- **pressure**- Diastolic blood pressure (mm Hg) (numeric);
- **triceps**- Triceps skin fold thickness (mm) (numeric);
- **insulin**- 2-Hour serum insulin (mu U/ml)(numeric);
- **mass**- Body mass index (weight in kg/(height in m)^2) (numeric);
- **pedigree**- Diabetes pedigree function (numeric);
- **age**- Age (years) (numeric);
- **diabetes**- class variable: diabetic or not (factor with 2 levels: neg and pos

```
dim(PimaIndiansDiabetes2)
```

```
## [1] 768   9
```

```r
head(PimaIndiansDiabetes2)
```

```
##   pregnant glucose pressure triceps insulin mass pedigree age diabetes
## 1        6     148       72      35      NA 33.6    0.627  50      pos
## 2        1      85       66      29      NA 26.6    0.351  31      neg
## 3        8     183       64      NA      NA 23.3    0.672  32      pos
## 4        1      89       66      23      94 28.1    0.167  21      neg
## 5        0     137       40      35     168 43.1    2.288  33      pos
## 6        5     116       74      NA      NA 25.6    0.201  30      neg
```

Data Cleaning

```r
#omit missing values
PimaIndiansDiabetes2 <- na.omit(PimaIndiansDiabetes2)
```

```r
dim(PimaIndiansDiabetes2)
```

```
## [1] 392   9
```

```r
head(PimaIndiansDiabetes2)
```

```
##    pregnant glucose pressure triceps insulin mass pedigree age diabetes
## 4         1      89       66      23      94 28.1    0.167  21      neg
## 5         0     137       40      35     168 43.1    2.288  33      pos
## 7         3      78       50      32      88 31.0    0.248  26      pos
## 9         2     197       70      45     543 30.5    0.158  53      pos
## 14        1     189       60      23     846 30.1    0.398  59      pos
## 15        5     166       72      19     175 25.8    0.587  51      pos
```

Data Splitting

```r
set.seed(123)
training.samples <- PimaIndiansDiabetes2$diabetes %>% createDataPartition(p=0.8, list=FALSE)
train.data <- PimaIndiansDiabetes2[training.samples,]
test.data <- PimaIndiansDiabetes2[-training.samples,]
```

Model fitting

```r
#fit the model
fit <- glm(diabetes ~., data=train.data, family = binomial)

#summary
summary(fit)
```

```
##
## Call:
## glm(formula = diabetes ~ ., family = binomial, data = train.data)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.053e+01  1.440e+00  -7.317 2.54e-13 ***
## pregnant     1.005e-01  6.127e-02   1.640  0.10092
## glucose      3.710e-02  6.486e-03   5.719 1.07e-08 ***
## pressure    -3.876e-04  1.383e-02  -0.028  0.97764
## triceps      1.418e-02  1.998e-02   0.710  0.47800
## insulin      5.940e-04  1.508e-03   0.394  0.69371
## mass         7.997e-02  3.180e-02   2.515  0.01190 *
## pedigree     1.329e+00  4.823e-01   2.756  0.00585 **
## age          2.718e-02  2.020e-02   1.346  0.17840
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 398.80  on 313  degrees of freedom
## Residual deviance: 267.18  on 305  degrees of freedom
## AIC: 285.18
##
## Number of Fisher Scoring iterations: 5
```

Model prediction

```
# find Y hat of each x
pred_prob <- fit %>% predict(test.data, type="response")
```

Confusion Matrix

From scratch

```
#create predicted class
predicted.classes <- ifelse(pred_prob > 0.5, "pos", "neg")

#create confusion matrix
table(predicted.classes,test.data$diabetes)
```

```
##
## predicted.classes neg pos
##               neg  44  11
##               pos   8  15
```

Built-in function

```
cm <- confusionMatrix(factor(predicted.classes),factor(test.data$diabetes), positive = "pos")
cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction neg pos
##        neg  44  11
##        pos   8  15
##
##                Accuracy : 0.7564
##                  95% CI : (0.646, 0.8465)
##     No Information Rate : 0.6667
##     P-Value [Acc > NIR] : 0.05651
##
##                   Kappa : 0.4356
##
##  Mcnemar's Test P-Value : 0.64636
##
##             Sensitivity : 0.5769
##             Specificity : 0.8462
##          Pos Pred Value : 0.6522
##          Neg Pred Value : 0.8000
##              Prevalence : 0.3333
##          Detection Rate : 0.1923
##    Detection Prevalence : 0.2949
##       Balanced Accuracy : 0.7115
##
##        'Positive' Class : pos
##
```

When positive class = "1" Detect positive

| Actual/Predicted | Negative (Actual 0) | Positive (Acutal 1) |

| Negative (Predicted 0) | **True Negative (TN)** | **False Negative(FN)** |

| Positive (Predicted 1) | **False Positive(FP)** | **True Positive(TP)** |

Correct:

- Out of 55 sample predicted "neg", 44 predicted "neg" correctly
- Out of 23 sample predicted "pos", 15 predicted "pos" correctly

Wrong:

- Out of 55 sample predicted "neg", 11 predicted "neg" incorrectly (should be "pos")
- Out of 23 sample predicted "pos", 8 predicted "pos" incorrectly (should be "neg")

Model Accuracy

Use R:

```r
#Model Accuracy
cm$overall["Accuracy"]
```

```
##  Accuracy
## 0.7564103
```

Manual Calculation:

$P(\text{predicted correctly}) = \frac{\text{num of sample predicted "neg" correctly+num of sample predicted "pos" correctly}}{\text{Total num of sample}}$

```
(44+15)/(44+11+8+15)
```

```
## [1] 0.7564103
```

Sensitivity

Use R:

```
cm$byClass["Sensitivity"]
```

```
## Sensitivity
##   0.5769231
```

$Sensitivity = P(\text{predicted positive}|\text{positive})$

Manual Calculation

```
sen <- 15/(11+15)
sen
```

```
## [1] 0.5769231
```

Specificity

Use R:

```
cm$byClass["Specificity"]
```

```
## Specificity
##   0.8461538
```

$Specificity = P(\text{predicted negative}|\text{negative})$

Manual Calculation

```
spec <- 44/(44+8)
spec
```

```
## [1] 0.8461538
```

F1 Score

```r
cm$byClass["F1"]
```

```
##        F1
## 0.6122449
```